

DATA 266 — Homework 2.5

Rajesh Paruchuri SID4 = 1605 | SEED = 1605 | SLICE = 605 | HP_ID = 3 | CLS_A = 5 | CLS_B = 3

HP_ID is reported only. No second hyperparameter model for this assignment.

All numbers below are filled in from the RTX 4090 run recorded in `METRICS.md` and `results/NVIDIA GeForce RTX 4090_5b052ad1`. Export this file to PDF for submission.

Step 0

Same standing parameters as assignment 1 (Section 0.1). The scripts and `hw2.5.ipynb` seed Python / NumPy / PyTorch with 1605. No extra HP_ID network is trained for HW2.5.

Part A — machine and provenance

I reserved an RTX 4090 workstation and logged the reservation and GPU-hours in `reservations/GPU_HOURS.md`. The full `nvidia-smi -q` dump is under `results/NVIDIA GeForce RTX 4090_5b052ad1/`.

NVIDIA GeForce RTX 4090	
UUID	GPU-5b052ad1-4272-40db-4b25-c930bf32b547
Driver	610.60
CUDA	13.3 (nvidia-smi) / 13.0 (torch)
VRAM	24564 MiB
Power limit	450.00 W
Architecture	Ada Lovelace (AD102)
Memory	24 GB GDDR6X, 384-bit, 1008 GB/s
Tensor cores	4th generation — TF32, FP16, BF16, FP8, INT8, INT4

Sources: NVIDIA Ada GPU Architecture whitepaper; NVIDIA RTX Blackwell GPU Architecture whitepaper, Table 3; the matching GeForce product page. All peaks are dense (no 2:4 sparsity) — see `vendor/GPU_SPECS.md`.

Part B — precision and achieved throughput

Figure: `figures/part_b_tflops_NVIDIA GeForce RTX 4090_5b052ad1.png`, one line per precision, $N = 1024/4096/8192/16384$.

On UUID GPU-5b052ad1-4272-40db-4b25-c930bf32b547: FP32 plateaus at $N=4096$ (53.24 TFLOPS, 95% of its own 54.61 TFLOPS max at $N=8192$). TF32 plateaus at $N=4096$ (85.60 TFLOPS, 95% of its 87.57 TFLOPS max at $N=8192$). FP16 plateaus at $N=4096$ (166.48 TFLOPS — its own max). BF16 plateaus at $N=8192$ (161.75 TFLOPS — its own max).

One anomaly worth flagging: TF32 measured 103.6-106.0% of the 82.6 TFLOPS dense-peak figure I used as the denominator (the same figure widely cited as the RTX 4090's non-tensor FP32 peak). Since measured throughput cannot exceed a correct peak, this means that 82.6 TFLOPS figure understates what Ada's tensor cores actually deliver in TF32 mode — the “% of peak” column for TF32 in this table should be read as approximate rather than a literal percentage of an authoritative ceiling.

None of the precisions plateau at $N=1024$: a 1024^3 GEMM does not launch enough thread blocks to saturate every SM, cuBLAS kernel-selection and launch overhead is a large share of the timed interval, and the operands are small enough to sit largely in cache, so the timer is measuring launch latency rather than the tensor cores or DRAM at steady state. Throughput only converges once N is large enough that the GEMM runs long enough to amortize that fixed cost.

% of theoretical peak uses the dense peak for that precision. FP16 and BF16 compare against the FP32-accumulate tensor-core column, because that is what PyTorch’s `torch.mm` actually executes for those dtypes on these cards.

Lower precision: I attempted FP8 through `torch._scaled_mm` and FP4 through whatever dtype the installed PyTorch build exposes. FP8 succeeded via `torch._scaled_mm(a8, b8.t().contiguous().t(), ...)` at N=8192, reaching 263.39 TFLOPS (79.7% of the 330.3 TFLOPS dense FP8 peak) — expected, since Ada’s 4th-generation tensor cores support FP8 in hardware. FP4 failed: the installed PyTorch exposes a `float4_e2m1fn_x2` dtype symbol at the software level, but there is no public dense GEMM path wired to it, and Ada (AD102) does not implement FP4 in its tensor cores in the first place (that arrives with Blackwell’s 5th generation), so this failure is expected on this card and is reported as a legitimate finding rather than a missing measurement.

Part C — bandwidth-bound vs compute-bound

The elementwise add (`out = x + y`) reads two float32 operands and writes one, so it moves 12 bytes per FLOP performed (arithmetic intensity = 1/12 FLOP/byte). The copy variant moves 8 bytes and performs zero FLOPs. Both sit far to the left of the ridge point for this card, i.e. on the memory-bound side of the roofline.

A square BF16 GEMM at large N has arithmetic intensity N/3 FLOP/byte, which for N=16384 is orders of magnitude above the BF16 ridge point (peak tensor TFLOPS divided by the card’s spec bandwidth), putting it firmly on the compute-bound side.

On UUID GPU-5b052ad1-4272-40db-4b25-c930bf32b547: the elementwise add reached 916.3 GB/s, 90.9% of the 1008.0 GB/s spec bandwidth (AI = 0.0833 FLOP/byte, memory-bound, left of the ridge). The copy check landed close by at 907.8 GB/s (90.1% of spec), consistent with the add measurement. The square BF16 GEMM at N=16384 reached 161.01 TFLOPS, 97.5% of the 165.2 TFLOPS BF16 peak (AI = 5461.3 FLOP/byte, compute-bound, far right of the ridge). Ridge points for this card: 81.9 FLOP/byte (FP32), 163.9 FLOP/byte (BF16).

These two ceilings — the memory ceiling and the compute ceiling — are exactly the two numbers this course keeps coming back to after Week 3, and my numbers confirm they do not coincide: achieved add bandwidth falls short of the GDDR spec number, and achieved GEMM throughput falls short of the tensor-core spec number, by different margins.

Part D — the cost of attention

Config, chosen once and stated here: batch size **1**, **16** heads, head dimension **128**, dtype bfloat16. The naive path builds the full [B, H, S, S] score tensor and its softmax before contracting with V; the fused path calls `torch.nn.functional.scaled_dot_product_attention`.

On UUID GPU-5b052ad1-4272-40db-4b25-c930bf32b547:

seq	naive latency (ms)	naive peak memory (GB)
512	0.10	0.034
1024	0.16	0.092
2048	1.00	0.311
4096	3.72	1.149
8192	14.50	4.438
16384	57.26	17.457

Naive OOM boundary: largest tested sequence length that succeeded = 37760, smallest that failed = 37824, a gap of 64 tokens. I am reporting both bounds and am not claiming a single-token boundary, since the gap is 64, not 1.

Quadratic fit (from my own measured naive peak-memory points, not asserted from the $O(S^2)$ argument alone): $\text{peak_bytes} \sim a \cdot S^2 + b \cdot S + c$, with $a = 64 \text{ bytes/token}^2$, $b = 16384 \text{ bytes/token}$, $c = 8,519,680 \text{ bytes}$. The quadratic fit’s RMSE ($3.22\text{e-}6$) is roughly 15 orders of magnitude smaller than the linear fit’s RMSE ($1.51\text{e}9$), which is as decisive a confirmation of the quadratic term as this data can give. The coefficients also match the theory exactly for my chosen config ($B=1$, $H=16$, $\text{head_dim}=128$, bf16): the S^2 term comes from two same-shaped $[B, H, S, S]$ tensors living briefly at once (raw scores plus their softmax), giving $2 \cdot B \cdot H \cdot 2 \text{ bytes} = 2 \cdot 1 \cdot 16 \cdot 2 = 64$; the linear term comes from the four $[B, H, S, D]$ tensors (Q , K , V , output), giving $4 \cdot B \cdot H \cdot D \cdot 2 \text{ bytes} = 4 \cdot 1 \cdot 16 \cdot 128 \cdot 2 = 16384$.

The S^2 term is the materialized attention matrix itself. The fused kernel never writes that matrix to HBM: it tiles Q , K , and V into on-chip SRAM, computes local score blocks, maintains a running (unnormalized) softmax across tiles, and writes only the final output tile back to HBM. That collapses the memory footprint from $O(S^2)$ to $O(S)$ in the sequence length, which is why its OOM wall sits much further out.

Fused OOM boundary: the fused kernel never failed up to the search cap of 131072 tokens (versus naive’s ~ 37760 -37824 boundary) — consistent with its $O(S)$ rather than $O(S^2)$ memory growth.

seq	naive (ms)	fused (ms)	speedup
512	0.10	0.13	0.75x
1024	0.16	0.36	0.45x
2048	1.00	1.22	0.82x
4096	3.72	3.25	1.15x
8192	14.50	5.27	2.75x
16384	57.26	20.41	2.81x

Notably, fused is *slower* than naive below $S \sim 2048$: the specialized kernel’s fixed per-call overhead isn’t amortized at short sequences, where PyTorch’s highly-tuned cuBLAS matmul + softmax path wins outright. The benefit only shows up once S is large enough that the $O(S^2)$ naive path’s extra compute and memory traffic dominate that fixed overhead — exactly the tradeoff documented for FlashAttention-style kernels in the literature.

Part E — sustained load and thermal behavior

20-minute BF16 GEMM at $N=8192$, `nvidia-smi` sampled every 5 seconds. Figure: graphics clock and temperature over time on one plot.

Throttling did occur, and I confirmed the mechanism directly from the raw `thermal_smi.csv` rather than trusting the summary label alone. The first sampled row ($t \sim 0$ s) shows the card idle: 210 MHz, 34°C , 15 W, `clocks_event_reasons.gpu_idle` active. From the very next full-load sample onward (detected at $t \sim 10$ s in the processed log), `clocks_event_reasons.sw_power_cap` is **active in every subsequent row**, with power draw pinned at 448-450 W against the card’s 450 W TDP limit, and the graphics clock oscillating between roughly 2505 and 2520 MHz rather than holding a higher boost clock. Critically, `hw_thermal_slowdown` and `sw_thermal_slowdown` stay **not active** throughout this window, and temperature only reached 55 - 62°C in the same rows (max 76°C over the full 20 minutes) — well short of the RTX 4090’s thermal throttle point. So this card is power-limited, not temperature-limited: it hits its 450 W ceiling almost as soon as the sustained BF16 GEMM load ramps up, and the SM clock backs off to stay inside that power budget rather than because the card is running hot.

Steady-state vs. peak throughput: first-30-second mean = 161.06 TFLOPS, last-5-minute mean = 157.44 TFLOPS, giving **steady/peak = 97.75%**. Since the power cap engages almost immediately, both windows are already power-limited — the further $\sim 2.25\%$ decline over 20 minutes is most likely temperature-driven softening of the voltage/frequency curve within that same 450 W budget, not a separate throttle event. This number is specific to my card, my chassis, and the room it ran in — it should not match anyone else’s.

A benchmark that runs for a few seconds only ever sees the card cold. The 20-minute number here is closer to what an actual training run in this course will look like.

Part F — Table HW2.5.1

Copied from METRICS.md after running `scripts/06_compile_report.py`. Every cell in that table carries the GPU UUID it was measured on.